

Inheritance

Inheritance

- Reusability is yet another feature of OOP's. C++ strongly supports the concept of reusability. The C++ classes can be used again in several ways. Once a class has been written and tested, it can be adopted by another programmers. This is basically created by defining the new classes, reusing the properties of existing ones. The mechanism of deriving a new class from an old one is called 'INHERTTENCE'.
- Inheritance allows us to define a class in terms of another class, which makes it easier to create and maintain an application. This also provides an opportunity to reuse the code functionality and fast implementation time.

Inheritance

- When creating a class, instead of writing completely new data members and member functions, the programmer can designate that the new class should inherit the members of an existing class.
- The old class is called '**BASE**' or **Parent** or **Super** class and the new one is called '**DERIEVED**' or **Child** or **Sub** class.
- The idea of inheritance implements the 'is a' relationship. For example, mammal IS-A animal, dog IS-A mammal hence dog IS-A animal as well and so on.

Benefits of Inheritance

- Inheritance promotes reusability. When a class inherits or derives another class, it can access all the functionality of inherited class.
- Inheritance enhance the reliability. The base class code will be already tested and debugged.
- As the existing code is reused, it leads to less development and maintenance costs.
- Inheritance helps to reduce code redundancy and supports code extensibility.
- Inheritance facilitates creation of class libraries.

Disadvantages

- Inherited functions work slower than normal function as there is indirection.
- Improper use of inheritance may lead to wrong solutions.
- Often, data members in the base class are left unused which may lead to memory wastage.
- Inheritance increases the coupling between base class and derived class. A change in base class will affect all the child classes.

Base and Derived Classes

- A class can be derived from more than one classes, which means it can inherit data and functions from multiple base classes. To define a derived class, we use a class derivation list to specify the base class(es). A class derivation list names one or more base classes and has following form:
class derived-class: **access-specifier** base-class
- Where access-specifier is one of **public**, **protected**, or **private**, and base-class is the name of a previously defined class. If the access-specifier is not used, then it is private by default.
- In the inheritance, some of the base class data elements and member functions are inherited into the derived class. We can add our own data and member functions and thus extend the functionality of the base class. Inheritance, when used to modify and extend the capabilities of the existing classes, becomes a very powerful tool for incremental program development.

Example

```
class xyz          //base class
{
members of xyz
};
class ABC : public xyz    //public derivation
{
members of ABC
};
class ABC : private xyz   //private derivation
{
members of ABC
};
class ABC : protected xyz //protected derivation
{
members of ABC
};

class ABC : XYZ //private derivation (by default)
```

Rules of Inheritance

- When deriving a class from a base class, the base class may be inherited through public, protected or private inheritance.
- We hardly use protected or private inheritance, but public inheritance is commonly used. While using different type of inheritance, following rules are applied –
- **Public Inheritance** – When deriving a class from a public base class, public members of the base class become public members of the derived class and protected members of the base class become protected members of the derived class. A base class's private members are never accessible directly from a derived class, but can be accessed through calls to the public and protected members of the base class.
- **Protected Inheritance** – When deriving from a protected base class, public and protected members of the base class become protected members of the derived class.
- **Private Inheritance** – When deriving from a private base class, public and protected members of the base class become private members of the derived class.

Access Control in Inheritance

- A derived class can access all the non-private members of its base class. Thus base-class members that should not be accessible to the member functions of derived classes should be declared private in the base class.
- We can summarize the different access types according to - who can access them in the following way –

Access	public	protected	private
Same class	yes	yes	yes
Derived classes	yes	yes	no
Outside classes	yes	no	no

Access Control in Inheritance

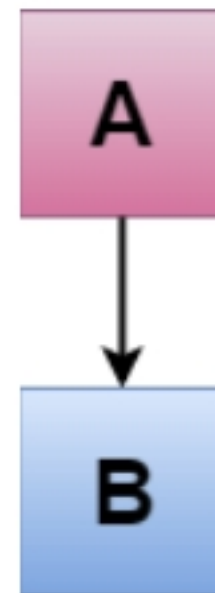
- A derived class inherits all base class methods except the following :
 - Constructors, destructors and copy constructors of the base class.
 - Overloaded operators of the base class.
 - The friend functions of the base class.

Types of Inheritance

- Single inheritance
- Multiple inheritance
- Hierarchical inheritance
- Multilevel inheritance
- Hybrid inheritance

Single inheritance

- Single inheritance is defined as the inheritance in which a derived class is inherited from the only one base class.



- Where 'A' is the base class, and 'B' is the derived class.

Example

```
#include<iostream>
using namespace std;
class worker
{
int age;
char name [10];
public:
void getworker();
void showworker ();
};
void worker :: getworker()
{
cout <<"Enter Name: ";
cin >> name;
cout <<"Enter Age: " ;
cin >> age;
}
void worker :: showworker ()
{
cout <<"Name: "<<name<<endl;
cout<<"Age: "<<age<<endl;
}
class manager : public worker
{
int now;
public:
void get () ;
```

```
void show () ;
};
void manager :: get ()
{
cout << "Enter Number of Worker under Manager:";
cin >> now;
}
void manager :: show ()
{
cout <<"Number of Worker under Manager: "<<
now<<endl;
}
int main ()
{
manager M1;
cout<<"Give Worker Details:\n";
M1.getworker();
M1.get();
cout<<"\nGiven Worker Details:\n";
M1.showworker();
M1.show();
return 0;
}
```

How to make a Private Member Inheritable

- The private member is not inheritable. If we modify the visibility mode by making it public, but this takes away the advantage of data hiding.
- C++ introduces a third visibility modifier, i.e., protected. The member which is declared as protected will be accessible to all the member functions within the class as well as the class immediately derived from it.

Base class visibility	Derived class visibility		
	Public	Private	Protected
Private	Not Inherited	Not Inherited	Not Inherited
Protected	Protected	Private	Protected
Public	Public	Private	Protected

Function overriding

- The same function is defined in both the derived class and the based class. Now if we call this function using the object of the derived class, the function of the derived class is executed.
- This is known as function overriding in C++. The function in derived class overrides the function in base class.

Example:

```
#include <iostream>
using namespace std;
class Base {
public:
    void print() {
        cout << "Base Function" << endl;
    }
};

class Derived : public Base {
public:
    void print() {
        cout << "Derived Function" << endl;
    }
};

int main() {
    Derived derived1;
    derived1.print();
    derived1.print();
    return 0;
}
```

Output:
Derived Function
Derived Function

Access Overridden Function

```
#include <iostream>
using namespace std;

class Base {
public:
    void print() {
        cout << "Base Function" << endl;
    }
};

class Derived : public Base {
public:
    void print() {
        cout << "Derived Function" << endl;
    }
};

int main() {
    Derived derived1;
    derived1.print();
    derived1.Base::print();
    return 0;
}
```

Output:
Derived Function
Base Function

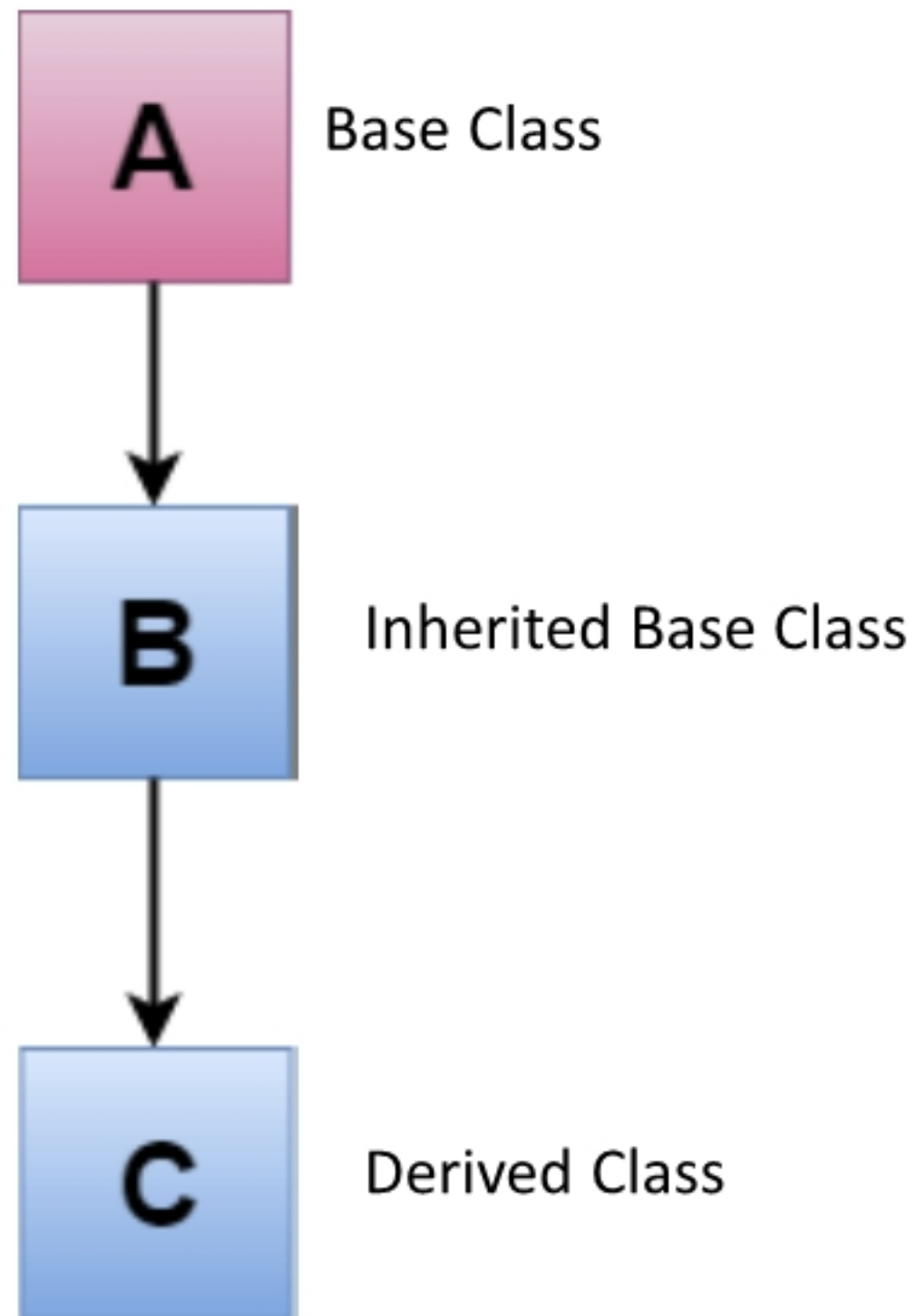
Multilevel Inheritance

- When one class inherits another class which is further inherited by another class, it is known as multi level inheritance in C++. Inheritance is transitive so the last derived class acquires all the members of all its base classes.
- For Example: the class A serves as a base class for a derived class B which in turn serves as a base class for the derived class C. This type of inheritance is called 'MULTILEVEL INHERITENCE'. The class B is known as the 'INTERMEDIATE BASE CLASS' since it provides a link for the inheritance between A and C. The chain ABC is called 'INHERITENCE PATH'

Multilevel Inheritance

The declaration Syntax
for Same Classes:

```
class A  
{  
    //body  
}  
class B : public A  
{  
    //body  
}  
class C : public B  
{  
    //body  
}
```



```

#include<iostream>
using namespace std;
class student
{
protected:
int rno,m1,m2;
public:
void get()
{
cout<<"Enter the Roll no :";
cin>>rno;
cout<<"Enter the two marks :";
cin>>m1>>m2;
}
};
class sports : public student
{
protected:
int sm; // sm = Sports mark
public:
void getsn()
{
cout<<"\nEnter the sports mark :";
cin>>sm;

```

Example:

```

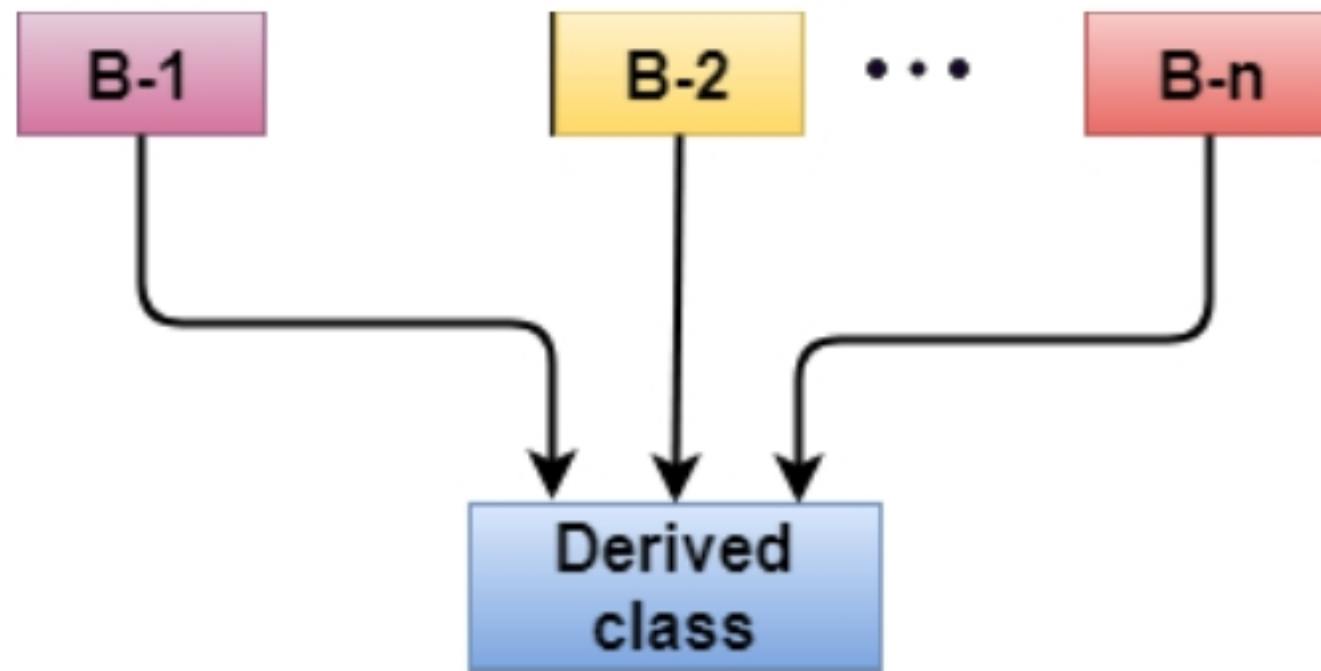
}
};
class statement : public sports
{
int tot,avg;
public:
void display()
{
tot=(m1+m2+sm);
avg=tot/3;
cout<<"\n\n\tRoll No : "<<rno<<"\n\tTotal : "<<tot;
cout<<"\n\tAverage : "<<avg;
}
};
int main()
{
statement std;
std.get();
std.getsm();
std.display();
return 0;
}

```

Multiple Inheritance

- A class can inherit the attributes of two or more classes. This mechanism is known as 'MULTIPLE INHERITENCE'. Multiple inheritance allows us to combine the features of several existing classes as a starting point for defining new classes.
- Syntax of the Derived class:
 class D : visibility B_1, visibility B_2,...
 {
 // Body of the class;
 }

Multiple Inheritance



Example:

```
#include<iostream>
using namespace std;
class father //Declaration of base classl
{
int age ;
char name[20] ;
public:
void get ( ) ;
void show ( ) ;
};
void father::get ( )
{
cout << "Enter Father name: ";
cin >> name;
cout << "Enter the age: ";
cin >> age;
} void father::show ( )
{
cout<< "Father's name is: "
<<name<<endl;
cout<<"Father's age is:"<<age<<endl;
}

class mother {
char name [20] ;
int age ;
```

```
public:
void get ( )
{
cout << "Enter mother name:";
cin >> name;
cout << "Enter the age:";
cin >> age;
}
void show ( )
{
cout<<"Mother name is: "
<<name<<endl;
cout<<"Mother age is: "<<age<<endl;
}
};

class child : public father, public mother //derived class inheriting
{
char name[20] ;
int std;
public:
void get 1( ) ;
void show ( ) ;
};
void child :: get 1( )
{
```

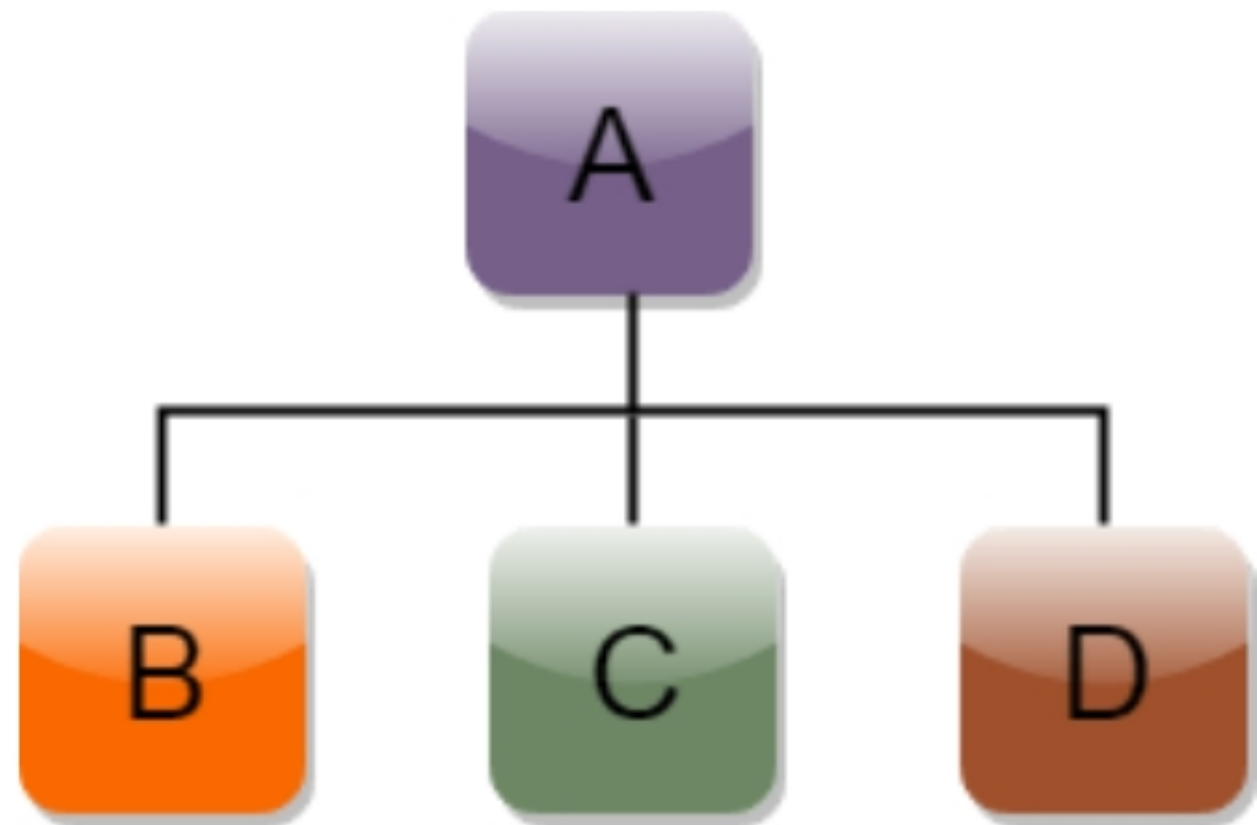
```
father::get() ;
mother::get() ;
cout<<"Enter child name:";
cin>>name;
cout<<"Enter child standard:";
cin >> std;
}
void child :: show ( )
{
father::show();
mother::show() ;
cout<<"Child name is : "
<<name<<endl;
cout<<"Child standard: "<<std<<endl;
}
main ( )
{
child C;
C.get1();
cout<<"Given Detail:.....\n";
C.show();
return 0;
}
```

Hierarchical Inheritance

- Hierarchical inheritance is defined as the process of deriving more than one class from a base class.

- Syntax:

```
class A
{
    // body of the class A.
}
class B : public A
{
    // body of class B.
}
class C : public A
{
    // body of class C.
}
class D : public A
{
    // body of class D.
}
```



Example:

```
#include<iostream>
using namespace std;
class Father //Declaration of base classl {
{
int age ;
char name[20] ;
public:
void get ( ) ;
void show ( ) ;
};
void Father::get ( )
{
cout << "Enter Father name: ";
cin >> name;
cout << "Enter the age: ";
cin >> age;
}
void Father::show ( )
{
cout<< "Father's name is:
"<<name<<endl;
cout<<"Father's age is:"<<age<<endl;
}

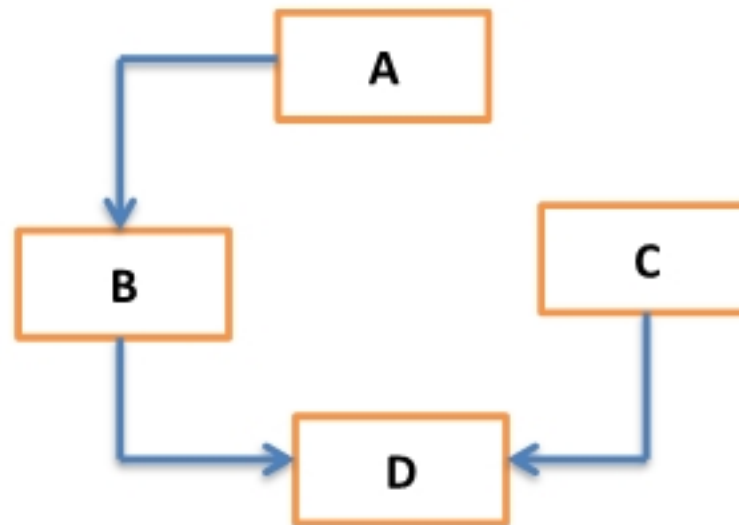
class Daughter:public Father
{
char name [20] ;
int age ;
public:
void get ( ) ;
void show ( ) ;
};

class Son : public Father
{
char name[20] ;
int age;
public:
void get ( ) ;
void show ( ) ;
};

void Son :: get ( )
{
Father::get() ;
cout<<"Enter Son name:";
cin>>name;
cout<<"Enter Son Age:";
cin >> age;
}
void Son :: show ( )
{
Father::show();
cout<<"Son name is : "<<name<<endl;
cout<<"Son Age is : "<<age<<endl;
}
main ( )
{
Son S;
Daughter D;
cout<<"Give Son Details:\n";
S.get();
cout<<"Give Daughter Details:\n";
D.get();
cout<<"\n\nGiven Detail of Son:\n";
S.show();
cout<<"\n\nGiven Detail of
Daughter:\n";
D.show();
return 0;
}
```

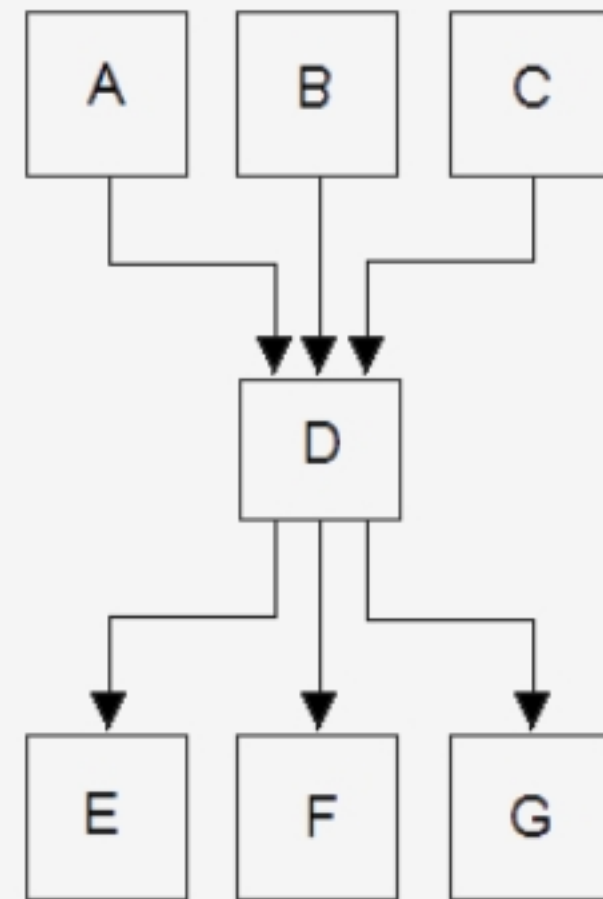

Hybrid Inheritance

- There could be situations where we need to apply two or more types of inheritance to design a program. Basically Hybrid Inheritance is the combination of one or more types of the inheritance.



Hybrid Inheritance

- Syntax:
- ```
class A
{
// BODY OF THE CLASS A
};
class B : public A
{
// BODY OF THE CLASS A
};
class C
{
// BODY OF THE CLASS A
};
class D : public B, public C
{
// BODY OF THE CLASS A
};
```



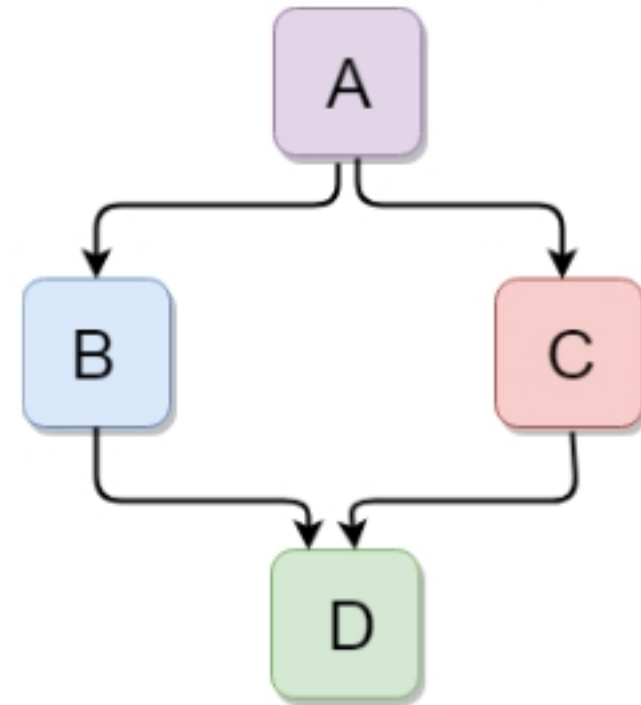
```

#include<iostream>
using namespace std;
class student //base class declaration
{
char name[40];
int id;
public:
void getdetail()
{
cout<<"Enter Name:";
cin>>name;
cout<<"Enter ID:";
cin>>id;
}
void putdetail()
{
cout <<"Name: "<<name<<endl;
cout <<"ID: "<<id<<endl;
}
};
class test : public student
{
protected:
int Sub1,Sub2;
public :
void getmarks()
{
cout<<"Enter Two Subject Mark: ";
cin>>Sub1>>Sub2;
}
void putmarks()
{
cout<< "Marks obtained: \n";
cout<<"Subject 1: "<<Sub1<<endl;
cout<<"Subject 2:"<< Sub2 <<endl;
}
};
class sports
{
protected:
int score;
public:
void getscore()
{
cout<<"Enter Sports Score:";
cin>>score;
}
void putscore()
{
cout << "Sports wt. :"<<score<<endl;
}
};
class result : public test, public sports
{
int total;
public:
void display();
}
void result::display()
{
total = Sub1 + Sub2 + score;
putdetail();
putmarks();
putscore();
cout<<"Total score: "<<total<<endl;
}
int main()
{
result std;
cout<<"Provide the Details of
Student:\n";
std.getdetail();
std.getmarks();
std.getscore();
cout<<"\n\n\t\tStudent Report
Card:\n\n";
std.display();
return 0;
}

```

# Virtual Base Class

- An ambiguity can arise when several paths exist to a class from the same base class. This means that a child class could have duplicate sets of members inherited from a single base class.
- C++ solves this issue by introducing a virtual base class. When a class is made virtual, necessary care is taken so that the duplication is avoided regardless of the number of paths that exist to the child class.



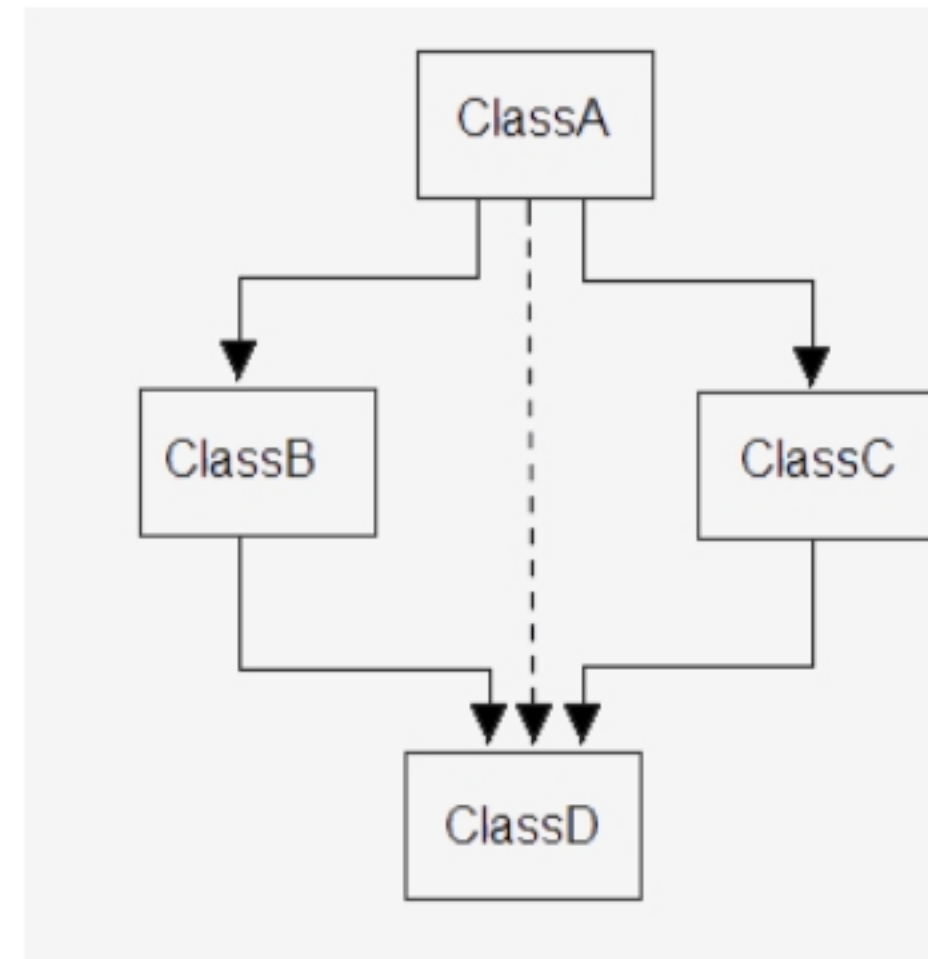
# Virtual Base Class

## Multi Path Problem

```
class A {
 //Body Of A
};
class B : public A
{
 //Body Of B
};
class C : public A
{
 //Body Of C
};
class D : public B, public C
{
 //Body Of D
};
```

## Solution using virtual base class

```
class A {
 //Body Of A
};
class B : public virtual A
{
 //Body Of B
};
class C : virtual public A
{
 //Body Of C
};
class D : public B, public C
{
 //Body Of D
};
```



```

#include<iostream>
using namespace std;
class student //base class declaration
{
char name[40];
int id;
public:
void getdetail()
{
cout<<"Enter Name:";
cin>>name;
cout<<"Enter ID:";
cin>>id;
}
void putdetail()
{
cout <<"Name: "<<name<<endl;
cout <<"ID: "<<id<<endl;
}
};
class test : public virtual student
{
protected:
int Sub1,Sub2;
public :
void getmarks()
{
cout<<"Enter Two Subject Mark: ";
cin>>Sub1>>Sub2;
}
void putmarks()
{
cout<<"Marks obtained: \n";
cout<<"Subject 1: "<<Sub1<<endl;
cout<<"Subject 2:"<< Sub2 <<endl;
}
};
class sports : public virtual student
{
protected:
int score;
public:
void getscore()
{
cout<<"Enter Sports Score:";
cin>>score;
}
void putscore()
{
cout <<"Sports wt. : "<<score<<endl;
}
};
class result : public test, public sports
{
int total;
public:
void display()
{
total = Sub1 + Sub2 + score;
putdetail();
putmarks();
putscore();
cout<<"Total score: "<<total<<endl;
}
}
int main()
{
result std;
cout<<"Provide the Details of
Student:\n";
std.getdetail();
std.getmarks();
std.getscore();
cout<<"\n\n\t\tStudent Report
Card:\n\n";
std.display();
return 0;
}

```

# Constructor in Derived Class

- While using constructors during inheritance, is that, as long as a base class constructor does not take any arguments, the derived class need not have a constructor function.
- However, if a base class contains a constructor with one or more arguments, then it is mandatory for the derived class to have a constructor and pass the arguments to the base class constructor.
- When both the derived and base class contains constructors, the base constructor is executed first and then the constructor in the derived class is executed.
- In case of multiple inheritance, the base class is constructed in the same order in which they appear in the declaration of the derived class. Similarly, in a multilevel inheritance, the constructor will be executed in the order of inheritance.
- Remember, Derived class can not inherit the base class constructor but it can call the base class constructor in derived class definition.



# Why is Base class Constructor called inside Derived class?

- Constructors have a special job of initializing the object properly. A Derived class constructor has access only to its own class members, but a Derived class object also have inherited property of Base class, and only base class constructor can properly initialize base class members. Hence all the constructors are called, else object wouldn't be constructed properly.



# Constructor in Derived Class

- The derived class takes the responsibility of supplying the initial values to its base class. The constructor of the derived class receives the entire list of required values as its argument and passes them on to the base constructor in the order in which they are declared in the derived class.
- Syntax:  

```
Derived_constructor(Arg1,Arg2..ArgN,Args):Base1(Arg1),Base2(Arg2),...BaseN(ArgN)
{
 //Body of Derived_Constructor with Args
}
```
- The header line of the derived-constructor function contains two parts separated by a colon (:). The first part provides the declaration of the arguments that are passed to the derived class constructor and the second part lists the function calls to the base class.
- Example:  

```
D(int a1, int a2, int b1, int b2, int d): A(a1, a2), B(b1, b2)
{
 d = d1;
}
```

# Example in Multiple Inheritance

```
#include<iostream>
using namespace std;
class alpha
{
 int x;
 public:
 alpha(int i)
 {
 x = i;
 cout << "\n alpha initialized \n";
 }
 void show_x()
 {
 cout << "\n x = " << x;
 }
};
class beta
{
 float y;
 public:
 beta(float j)
 {
 y = j;
 cout << "\n beta initialized \n";
 }
 void show_y()
 {
 cout << "\n y = " << y;
```

```
 }
 };
 class gamma : public beta, public alpha
 {
 int n,m;
 public:
 gamma(int a, float b, int c, int d): alpha(a), beta(b)
 {
 m = c;
 n = d;
 cout << "\n gamma initialized \n";
 }
 void show_mn()
 {
 cout << "\n m = " << m;
 cout << "\n n = " << n;
 }
 };

 int main()
 {
 gamma G(5, 7.65, 30, 100);
 cout << "\n Given Data: \n";
 G.show_x();
 G.show_y();
 G.show_mn();
 return 0;
 }
```

# More Example:

```
class Base
{
 int x;
 public:
 // default constructor
 Base()
 {
 cout << "Base default constructor\n";
 }
};

class Derived : public Base
{
 int y;
 public:
 // default constructor
 Derived()
 {
 cout << "Derived default constructor\n";
 }
};

// parameterized constructor
Derived(int i)
{
 cout << "Derived parameterized
constructor\n";
};

int main()
{
 Base b;
 Derived d1;
 Derived d2(10);
}
```





# Constructor and Destructor in Derived Class

- When an object of the derived class is created, the compiler first call the base class constructor and then the constructor of the derived class. This because the derived class is built up on the members of the base class. When the object of a derived class expires first the derived class destructor is invoked followed by the base class destructor.
- The constructors are executed in the order of inherited class i.e., from base constructor to derived. The destructors are executed in the reverse order.

# Example: Order of Execution

```
#include<iostream>
using namespace std;
class base
{
public:
base()
{
cout<<"\nConstructor of base class...";
}
~base()
{
cout<<"\nDestructor of base class.... ";
}
};
class derived:public base
{
public:
derived()
{
cout<<"\nConstructor of derived ...";
}
~derived()
{
cout<<"\nDestructor of derived ...";
}
};
class derived1:public derived
{
public:
derived1()
{
cout<<"\nConstructor of derived1 ...";
}
~derived1()
{
cout<<"\nDestructor of derived1 ...";
}
};
int main()
{
derived1 X;
return 0;
}
```